01 / Three-day finish plan

23-25 October 2026 | Asia/Shanghai | P1-P5 integration and submission readiness

Goal: finish one reproducible local banking demo with the real team modules connected, verified approvals/MFA, and submission evidence. Shared online hosting is optional and deferred.

What each person has completed

Role / progress	Completed in the reviewed code	What keeps the role unfinished
P1 - Claine ~85%	Five intents; Gemini/mock providers; typed proposals; transfer, spending and cancellation memory; grounded summaries; safe failure messages/logs; task switching; backend bridge and tests.	End-to-end real-module rehearsal, execution-result/state cleanup, UI metadata agreement and a bounded planning demonstration.
P2 - Marvel ~75%	FastAPI/DB, chat/read loop, action records/digests, transfer executor, React dashboard and approval/MFA views; Docker and fixture/demo seeding.	Real P4/P5 bridges, stored subscription status, execution controls/concurrency, metadata transport, schema upgrade and reproducible packaging.
P3 - Finance + MFA ~85%	Real finance bridge; Decimal/month/category/anomaly analytics. Random OTP, expiry, attempt limit, owner-scoped demo inbox, consumed challenge and HTTP/concurrency tests.	Review transaction boundaries with P2/P4, test the real policy path, check browser MFA errors and sign off report inputs.
P4 - Security ~80%	Pure fail-closed policy; strict arguments, ownership, lock, cumulative tiers, malformed config and injection/precision tests.	Backend still selects mock policy; execution-time re-evaluation, daily reset/locking, safety-lock semantics and risk evidence need integration.
P5 - Subscriptions ~80%	Recurring detector, cadence/price/confidence/evidence logic; eligibility and owner-scoped stored-record mapping/enrichment with tests.	Backend still selects mock detector; connect mapping, persist cancellation state and verify list/cancel/repeat flows in app.

How to read these percentages

Planning estimates for each full role, including integration and delivery; allow roughly +/-10 points. They are not test coverage or measured hours. P4/P5 have substantial standalone work, but integration is not complete simply because their unit tests pass.

Review basis: local feature/agent at commit 632ea84, clean working tree before this document; recent P3/P4 merges are present. No remote fetch was performed and unpushed teammate changes are not visible. Percentages reflect this snapshot, not unseen work.

02 / Current evidence and priorities

Fix the remaining connections before adding more features

Fresh baseline: 460 Python tests passed, with one existing google-genai/Pydantic deprecation warning. This run covered contracts, agent, backend, finance, subscriptions and security. It used an isolated SQLite database and made no live LLM request.

Priority	Finding in current code	Owner and required result
1	adapters.py selects SecurityAdapterService(use_mock=True) and SubscriptionsAdapterService(use_mock=True); even their non-mock branches return mocks.	**P2 + P4/P5:** wire actual functions, make mode selection explicit, and prove the routes reach them.
1	Cancellation executor writes an audit entry, but no subscription table/status is updated. Chat context still hardcodes Bob and iQIYI.	**P2 + P5:** owned stored records, active/support checks and a persistent simulated cancelled state.
1	Yellow confirmation has no fresh lock/policy check; Red verification checks lock but has a TODO for real policy. Transfer read/modify/write and Yellow claim need concurrency proof.	**P2 + P4:** recheck current rules and atomically claim/execute each action; P3 reviews MFA transaction boundaries.
2	Chat places action_id/digest/tier inside strict tool arguments; actions.py defines create_action_record twice. Approval results do not currently re-enter P1 through a correlated result loop.	**P1 + P2:** preserve canonical arguments, choose explicit UI metadata transport, remove duplicate implementation and consume pending P1 results.
2	Daily transfer total increments but no calendar reset is visible. Startup uses create_all only and skips all seed additions if any user exists.	**P2 + P4:** define Asia/Shanghai day handling and safe DB upgrade/seed policy without erasing existing data.

Competition requirements that still need evidence

The brief explicitly asks for vague/omitted intent, ordered task planning (DAG), interruption/rollback/human takeover, audit and safety lock. Current guided separate requests and unfinished-draft switching are useful, but do not prove a general planner or execution rollback. On Day 1 agree a bounded supported two-step plan and honest limits; do not quietly label a guided conversation a full DAG.

Frontend production build, fresh Docker build and live Gemini availability were not revalidated in this review. Browser examples from prior testing are encouraging, not a substitute for rerunning the updated commit. No claim of production security or real bank integration.

03 / Three days, clear checkpoints

Use these as relative work blocks if the team starts after 23 October | 6-7 focused hours per day

Day	Work blocks and owners	Checkpoint before moving on
Day 1 Connect	**0:00-0:30 All:** same commit, modes and dataset; delegate shared files. **0:30-2:30 P2+P4:** real policy + execution controls. **P5:** mapping contract/tests. **P3:** finance/MFA review. **P1:** result/UI cases and bounded plan design. **2:30-4:30 P2+P5:** real detector and owned subscription persistence. P1 checks summaries and canonical result envelopes. **4:30-6:00 pairs:** strict metadata/UI contract and migration/seed decisions. **6:00-7:00 All:** narrow PRs, combined regression, fix bridge failures.	**G1:** actual P1/P3/P4/P5 calls are observable; subscription reads are scoped; candidate vs verified status is preserved; one agreed local startup path works.
Day 2 Prove	**0:00-1:00 All:** merge G1 and rerun baseline. **1:00-3:00 P2+P3+P4:** Yellow/Red controls, duplicate/racing actions and current policy. **P1+P5:** list/cancel/result round trips. **3:00-4:30 P1+P2:** bounded ordered plan, explicit independent approvals and interruption; P5 verifies cancelled state. **4:30-6:00 All:** browser acceptance matrix and a small Gemini 3.5 Lite smoke test. **6:00-7:00:** repair failing critical paths; save evidence.	**G2:** no premature debit; exactly one execution; cancellation persists; wrong/expired OTP and lock changes fail safely; full guided flow plus bounded plan is demonstrated honestly.
Day 3 Package	**0:00-1:30 P2+one teammate:** clean local and Docker startup; database upgrade/preservation check. **1:30-3:00 All:** final combined tests and browser rehearsal on one merged commit. **3:00-4:30 P1+P2 lead:** architecture/README, <=200-word brief, security self-assessment, <=10-minute deck and <=5-minute video. **4:30-6:00:** record final demo and verify submission files. **6:00-7:00:** freeze candidate; critical fixes only, rerun affected checks.	**G3:** another teammate can start and rehearse without manual patches; code/tests/docs/video reference the same commit; limitations and provider mode are explicit.

Working agreement

P2 coordinates adapters.py, chat.py, actions.py, approvals.py and frontend changes. P3 owns delegated MFA files; P4 reviews controls; P5 owns mapping semantics. Use personal branches and short PRs. One edit window per shared file; never have all five push to feature/app.

Defer shared hosting, additional providers, visual redesign, extra scenarios and speculative latency work. If delayed, protect real bridges, safe execution, one persistent cancellation, three scenario demonstrations and required submission materials. Keep any unfulfilled challenge requirement explicit.

04 / P1 - finish the agent integration

Claine | ~85% now | partners: P2 for orchestration/UI, P3 for finance/MFA outcomes, P5 for subscription truth

Already completed

Five supported intents; configurable Gemini and deterministic mock; strict proposal/result validation; grounded balance/report replies; caller-owned clarification state and user/conversation isolation; safe logging/failure categories; friendly unknown-intent reply; explicit task switching and unmatched-subscription choices.

Day 1 - prove boundaries, not more phrase patches

- **1. Pair with P2 on canonical data.** Keep process_message and AgentResult unchanged. Agree a separate action-metadata representation for the UI; if frozen DTOs must change, use the contract maintainer and a versioned change. Test the returned arguments with validate_tool_arguments.

- **2. Finish result continuation design.** After confirmation/verification, P2 supplies the original conversation/proposal/tool/arguments plus trusted ExecutionResult. P1 handles and consumes completed/failed results, retaining pending MFA appropriately. Persist a user+conversation mapping; never infer outcomes from button clicks.

- **3. Design a bounded ordered plan with P2.** A supported example is detect/list -> select verified subscription -> independently approve cancellation -> request a specified transfer. Represent steps/dependencies explicitly, pause at missing fields and approvals, and never transfer projected savings automatically. Confirm intended requirement coverage with the team.

Day 2 - browser and bounded planning proof

- Implement the agreed small supported plan without general autonomous execution. Show pause/abandon/handoff before execution. Explain that cancelling a pending draft is not reversing an executed debit; any compensation needs a new approved action. Re-evaluate conditions before each mutation.

- Rehearse: balance; spending -> invalid month -> valid month; list -> unknown name -> verified name; transfer -> invalid amount -> valid amount; cancellation -> new balance request; pending proposal -> never mind. Check that prior issued tickets survive draft switching.

- Run a few live Gemini initial requests using an actually accepted key and gemini-3.5-flash-lite. Record provider failures separately from validation failures. Keep real P1 plus AGENT_PROVIDER=mock available for deterministic integration tests; do not call that live AI.

Day 3 - deliver the demonstration

Lead the demo script and architecture explanation. Show at least transfer, spending and subscriptions. Verify transcript, screenshots and video use the same final commit and data. Do not spend this day on a new provider or broad memory redesign.

Done when

P2 can run the agent from HTTP and UI; canonical tool arguments remain clean; approved outcomes are grounded and correlated; drafts/owners stay isolated; bounded plan scope and limitations are demonstrated; all P1-related tests pass on the shared commit.

Primary files: agent/service.py, conversation.py, request_guard.py, provider adapter and agent/tests/. Shared edit points with P2: backend/app/api/chat.py, approvals.py, session_store.py and frontend/src/api.ts/App.tsx. Shared hosting is not a P1 completion prerequisite.

05 / P2 - finish the app connections

Marvel | ~75% now | integration coordinator for all owners

Already completed

Database/read APIs, real P1 chat/state/read continuation, action storage/digests, balance/ledger/audit executor, React chat/approval/MFA inbox integration, Docker packaging and seed preservation. Fixture users/accounts now coexist with the Alice demo profile on a fresh database.

Day 1 - remove the remaining hidden mocks

- **1. With P4:** implement the real security.policy.evaluate_action branch and select it deliberately. With P5: connect detect_subscriptions and mapping/enrichment. Prove both chat and dashboard endpoints use actual functions; a changed flag with a mock fallback is not enough.

- **2. With P5:** add user-owned subscription storage/status and a reviewed seed strategy. Replace hardcoded chat subscriptions with stored records. Keep detected candidates separate from active supported records; no automatic promotion from confidence.

- **3. With P1:** separate UI tracking fields from strict tool arguments; persist proposal/action/user/conversation linkage and route trusted terminal results back to P1. Remove the duplicate create_action_record definition only after its final behavior is covered.

- **4. Database upgrade:** create_all does not add P3 MFA columns to existing tables. Supply a safe migration/upgrade step and test a fresh DB plus an older DB. Seed must not reset balances; decide whether to insert missing IDs instead of skipping the entire database.

Day 2 - close mutation controls

- With P4, validate canonical arguments again at execution, whitelist tools, recheck current ownership/lock/policy, define Asia/Shanghai daily-total reset and prevent two pending Yellow actions bypassing the cumulative MFA threshold.

- Use a database-backed atomic action transition and account/balance protection. Prove Yellow confirm races, Red verify races, repeated requests and insufficient funds produce one debit or none. Review the committed MFA-consumed marker and subsequent executor transaction with P3; avoid unrecoverable pending states.

- Persist cancellation status and return a consistent audit ID. Subsequent chat/context/listing and repeat cancellation must reflect the stored state. Pair with P1 on ordered plan/result routing and UI action status; distinguish draft abandonment from ticket cancellation.

Day 3 - make startup repeatable

Choose one recommended Docker path and keep the two-terminal path accurate. Verify Node/Vite compatibility and npm run build, backend dependencies, relative /api proxy, provider settings and secret handling. Ask another teammate to start from a fresh checkout and disposable database, then restart without losing data.

Done when

All four real modules are reached through the app; database updates are owned, durable and concurrency-safe for the demo; startup works without manual patches; frontend states match endpoints; README and recorded commit/modes are accurate.

Primary files: backend/app/services/adapters.py, actions.py, executor.py; api/chat.py, approvals.py, subscriptions.py; db/models.py and seed.py; frontend/src/api.ts and App.tsx; infra/ and docker-compose.yml. P2 coordinates these shared files; no unreviewed contract changes.

06 / P3 - verify finance and MFA

Finance + OTP/MFA | ~85% now | partners: P2 for executor/UI, P4 for controls, P1 for report grounding

Already completed

FinanceAdapterService now calls the real analyze_spending by default. Analytics cover exact Decimal debit totals, categories, anomaly history, local-month rules and diagnostics. MFA generates random six-digit codes, has five-minute challenge expiry bounded by action expiry, three failed attempts, owner-scoped simulated inbox, consumed marker and HTTP/concurrent tests.

Day 1 - preserve the working implementation

- **Finance:** sign off selected-account input, timezone-aware timestamps, requested month plus historical transactions, and no refund netting. Both fixture and Alice demo data are present but belong to different accounts. Supply expected outputs for the actual browser account.

- **MFA:** review issue/verify transaction boundaries with P2/P4. In current code the challenge is committed as consumed before execution. Define a safe failure/recovery state if execution or commit fails afterward; retries must not debit twice or leave the UI indefinitely pending.

- Check old database upgrades with P2: mfa_issued_at, mfa_expires_at, mfa_consumed_at and audit_id need actual columns. Do not assume create_all performs a migration. Confirm demo inbox is clearly simulated and owner-scoped under the demo identity model.

Day 2 - prove controls through the actual policy path

- Re-run: Red chat and confirm cause no debit; valid OTP causes exactly one debit; wrong code, third failure, expired challenge, expired action, foreign user and newly locked account do not execute. Preserve the original audit ID on successful replay.

- Use real P4 policy with current daily totals and balances; test two pending actions whose combined value crosses the threshold. Repeat concurrent verification against PostgreSQL as well as the existing SQLite checks with P2; report any untested database behavior explicitly.

- Pair with P2 on browser code entry, demo inbox visibility, expiry/attempt errors and disabled completed buttons. Pair with P1 on grounded MFA-pending/completed result replies; pending MFA must not be described as a successful transfer.

Day 3 - report and evidence handoff

Provide one verified October report, one actual anomaly example and a short finance/MFA explanation for the deck. Record OTP demo steps and known limits: no phone/email delivery, demo identity header, and no claim of production MFA. Contribute test names/results and review the video.

Data that must not be confused

Dataset	Expected before extra demo transfers
fixtures / account_001	October 2026 spending CNY 1843.00; no October anomalies. Credits/refunds are excluded, not subtracted.
Alice / acc_cny_001	Fresh seeded balance CNY 15250.00; October spending CNY 80.00; September CNY 45.00. Prior persisted transfers change balances and period totals.
P3 synthetic September fixture	CNY 9254.70 and seven full flags; label the dataset. Do not attribute it to Alice unless that dataset is deliberately seeded and scoped.

Primary files: finance/, backend/app/services/mfa.py, api/demo_inbox.py, delegated portions of api/approvals.py, and backend/tests/test_mfa.py. P4 specifies/reviews policy; P2 owns money execution. Do not replace working finance calculations while integrating MFA.

07 / P4 - connect and verify security

Security | ~80% now | partners: P2 for execution, P3 for MFA, P1 for canonical proposals

Already completed

Fail-closed pure evaluate_action; strict shared tool argument validation; owner/account matching; locked-account mutation rejection; Green/Yellow/Red cumulative transfer rules; invalid configuration, injected fields, malformed money and fixture-driven tests. The app currently still calls the mock policy adapter.

Day 1 - real bridge and agreed rules

- Pair with P2 to call the real evaluator in the backend and reject unknown tools. Add an integration test proving the actual evaluator is invoked, rather than testing only the standalone function or setting a flag.

- Define the execution-time BankingContext from current authenticated user/account, balance, lock and today's cumulative transfers. Validate the original source account instead of silently replacing it. Keep strict tool arguments free of action metadata.

- Agree day boundaries/reset with P2. Boundaries to demonstrate on a controlled database: 0+1000 is Yellow; 0+1000.01 is Red; 800+300 is Red. Two stale Yellow proposals cannot both bypass updated cumulative policy.

Day 2 - safety behavior and replay

- Review both Yellow confirm and Red verify. A correct digest or OTP cannot override a newly locked account, expired action, revoked subscription support, insufficient balance or a changed policy decision. P3 owns OTP mechanics; P2 performs state transitions/execution.

- With P2, test simultaneous confirmation/verification and competing transfers against one account. Require one execution per action, protected balances and consistent audit linkage. Existing MFA race tests are valuable but do not prove Yellow/action/account races generally.

- Agree and test the challenge brief's repeated-failure/suspicion safety lock. Distinguish failing one OTP action from setting the user/account lock. Define what triggers it, what operations remain readable, how a human/demo operator restores access, and audit each transition. Do not lock users merely because Gemini is unavailable.

- Check malformed/extra tool fields, cross-user action IDs, bad digests, merchant/note prompt injection and unknown tools. Read-only operations may remain allowed under lock according to the agreed policy. No untrusted report content becomes an instruction.

Day 3 - security self-assessment

Lead the required security document: implemented tiers, confirmation/MFA flow, injection defense, correlation/audit, replay/concurrency evidence, safety lock and known risks. Verify claims against the final merged app, not only policy unit tests. Review demo identity headers, console OTP visibility and local demo credentials as explicit prototype limits.

Done when

Real policy runs through HTTP and is re-evaluated before execution; boundary/race/lock tests pass; no duplicated debit or unauthorized cancellation; rejection and audit reasons are accurate; the self-assessment records remaining limitations honestly.

Primary files: security/ and its tests; delegated reviews/patches in adapters.py, actions.py, approvals.py and executor.py. Avoid introducing real bank connectivity or executing LLM-generated code. If arbitrary generated-code execution is absent, document that boundary instead of claiming a sandbox exists.

08 / P5 - finish subscription truth

Subscriptions | ~80% now | partners: P2 for storage/execution, P1 for explanation, P4 for authorization

Already completed

Real recurring detector with cadence/amount grouping, duplicate handling, confidence and evidence IDs; pure cancellation eligibility; mapping.py with owner-scoped ID/name/price resolution, ambiguity rejection, stored active/support checks and candidate enrichment. These are not yet connected to the backend's selected subscription service.

Day 1 - connect calculations to owned records

- **With P2:** replace mock fallback in both /subscriptions and chat list_subscriptions. Use same selected-account history; do not merge two accounts into a recurrence accidentally. Current dashboard route gathers all of the user's accounts, while chat uses a selected account; agree consistent scope.

- Agree an owned stored Subscription representation and ID mapping. Use match_stored/enrich_candidates/assess_stored_cancellation as appropriate. Candidate ID may differ from the stored backend ID; never assume a name alone is unique across users or services.

- Raw detector cancellation_supported=False is intentional. Enrichment may set it only for one owned active supported record. Confidence is recurrence evidence, not active status, permission or cancellation support. Confirm reason codes with P2/P1 before relying on them.

Day 2 - make cancellation durable

- With P2, persist simulated cancelled status after explicit confirmation. No current-month refund/credit is created and the balance does not increase. Refresh trusted chat context and UI status; historical recurring charges remain evidence even after cancellation, so do not simply erase the detector candidate.

- Verify unsupported, unknown, inactive and ambiguous records produce no executable proposal. Cancel once, re-list, retry cancellation and restart the app. The stored status must remain cancelled and repeat requests must have a defined safe result.

- Pair with P1 on unmatched names and available eligible choices. Test iQIYI for Alice's backend records separately from StreamBox/CloudMusic in shared fixtures; do not claim Netflix exists. No need for P5 to redesign the frontend; P2 implements UI and P5 validates meaning.

Day 3 - supply demo and report evidence

Provide a compact expected-output set for real detection/mapping and the cancellation walkthrough. Review screenshots/video for candidate-vs-active wording, supported/cancelled state and correct stored IDs. Document limited history/confidence and simulated merchant cancellation.

Acceptance checks with partners

Case	Expected
List subscriptions	Actual detector called with scoped history; evidence/cadence/confidence preserved. Alice's two iQIYI charges imply recurrence evidence, not proof of active status.
Unknown / unsupported name	Clear response and verified choices; no mutation proposal and no inferred support.
Confirm a supported cancellation	Stored status changes once, audit ID is linked, no balance credit, P1 receives trusted outcome.
Repeat / restart / another user	Cancelled state survives restart; no duplicate cancellation; no foreign-user record exposure.

Primary files: subscriptions/detector.py, eligibility.py, mapping.py and tests. P2 owns DB/API/executor changes; P1 owns language/grounding. Shared frozen contracts require coordinated versioning if the UI needs active/cancelled status not present in SubscriptionCandidate.

09 / Release gates and submission

One merged commit | one chosen dataset | real modules | explicit prototype limits

Gate / lead	Evidence required by end of Day 3
Modules / P2 + owners	Real P1, P3, P4, P5 functions are selected. Mock LLM interpretation is allowed only when labelled deterministic. Record modes and actual model; CLI/Docker defaults currently still use Gemini 3.1 Lite unless overridden.
Conversation / P1	Balance; month and transfer clarification; task switch; unknown subscription; wrong result correlation; separate users/conversations; trusted execution result continuation. Canonical arguments pass shared validators.
Mutations / P2-P4	Yellow confirm and Red MFA: no early debit, current policy/lock/expiry respected, same action cannot debit twice, account races protected, audit IDs consistent. Daily threshold/reset and safety lock demonstrated.
Subscriptions / P5-P2	Real detection plus verified mapping; cancellation persists and repeats safely; no projected savings credited. Listing alone never grants cancellation permission.
Startup / P2 + verifier	Fresh DB and existing DB upgrade tested; data preserved on restart; frontend production build and Docker startup work. Another teammate starts the final commit from README.

Submission owners - from the supplied competition brief

P1 + P2: project name and <=200-word brief; system architecture and bounded task-planning design. **P3:** analytics/MFA design and examples. **P4:** security self-assessment and known risks. **P5:** detector/mapping/cancellation design. **P2 coordinates:** source/GitHub, README, deployment instructions and test cases. **All:** <=10-minute defence deck and <=5-minute demo video; P1/P2 assemble and each owner checks claims.

Suggested five-minute video

0:00-0:30 problem, architecture and modes; 0:30-1:15 spending clarification/report; 1:15-2:15 real subscription detection and verified simulated cancellation; 2:15-3:30 transfer confirmation and Red OTP; 3:30-4:15 bounded ordered plan/interruption and one rejection; 4:15-5:00 audit, test evidence and limits. Use controlled starting totals; show independent approvals and no fabricated savings credit.

Reproduce the Python baseline

From repo root, PowerShell; uses an isolated test database, not the app DB:

```
$env:PYTHONDONTWRITEBYTECODE = '1'
$env:PYTHONPATH = (Get-Location).Path + ';backend'
$env:DATABASE_URL = 'sqlite:///:memory:'
agent/.venv/Scripts/python.exe -m pytest -p no:cacheprovider tests/test_contracts.py agent/tests backend/tests finance/tests subscriptions/tests security/tests -q --tb=short
```

Do not leave the test DATABASE_URL active when starting the real app. Final tests must be rerun after the integration changes; 460 passed is this review's baseline, not the future release result. Add targeted integration tests for the previously mocked bridges and execution gaps.

Freeze rule

Publish one release commit, startup/mode instructions, expected dataset values, test/build results, required artifacts and named remaining risks. Shared hosting is not explicitly required by the supplied brief; confirm the submission portal separately. Do not delay the demo for hosting, additional models or styling.